

# AIAIAI STORE

Sistema di Gestione di un Negozio Online - Enrico Lanzarolo

## Introduzione

Il progetto AIAIAI STORE è una piattaforma di e-commerce adibita alla vendita di abbonamenti mensili ad agenti di Intelligenza Artificiale utilizzabili in diversi ambiti (Marketing, Analisi Dati, Automazione, Assistenza Clienti, Sviluppo). La scelta di sviluppare un negozio di prodotti digitali ha richiesto la progettazione di un sistema in alcuni tratti diverso rispetto alla traccia assegnata, adattando ogni requisito alla logica degli abbonamenti.

Il sistema è stato sviluppato in PHP con logica procedurale rispettando i vincoli della consegna. Tra le varie caratteristiche l'architettura copre:

- database relazionale MySQL (aiaiai\_store.sql)
- pagina di connessione al db (connessione.php)
- pagina di login e registrazione con hashing delle password (login.php, registrazione.php)
- funzionalità di logout (logout.php)
- pagina home (index.php)
- pagina con carrello gestita tramite sessioni (carrello.php)
- pagina conferma ordine e riepilogo ordine (conferma\_ordine.php)
- storico abbonamenti con gestione degli stati (storico\_ordini.php. annulla\_abbonamento.php)
- pannello amministrativo solo per utenti con ruolo admin (index\_admin.php)
- pagine per visualizzazione dei prodotti (catalogo.php)
- pagina con dettaglio dei singoli prodotti (prodotto.php)

Il tutto accompagnato da un'interfaccia responsiva anche per telefono e dark-mode con design moderno garantita tramite i vari file css.

La struttura del progetto totale comprende quindi 12 file PHP, un file SQL per la creazione e il popolamento del database, 9 fogli di stile CSS specifici per pagina e un set di video per i background animati della home.

## Progettazione del Database Relazionale

La progettazione del database ha visto la modifica di alcuni vincoli della traccia e ciò è stato dovuto per via della natura del prodotto venduto. Alcune aggiunte/cambiamenti sono state le seguenti:

- Eliminazione del campo quantità\_disponibile dalla tabella prodotti: un'agente AI è un prodotto digitale con disponibilità illimitata e dunque questo campo non era funzionale.
- Sostituzione dello stato ordine (in elaborazione / spedito / consegnato) con stato\_abbonamento ENUM('attivo','annullato'): i prodotti digitali non hanno ciclo logistico; il servizio si attiva immediatamente all'acquisto e rimane tale fino a scadenza o cancellazione volontaria.

- Aggiunta del campo fascia\_abbonamento ENUM('base','pro','enterprise') nella tabella prodotti: classifica i tre livelli di ogni categoria e abilita la logica di upgrade.
- Aggiunta del campo ruolo ENUM('utente','admin') nella tabella utenti: differenzia i permessi di accesso tra clienti standard e amministratori.
- Aggiunta della tabella carte\_salvate: consente agli utenti di memorizzare metodi di pagamento e riutilizzarli negli acquisti successivi.
- Aggiunta dei campi data\_acquisto e data\_rinnovo nella tabella Ordini: essenziali per la gestione temporale degli abbonamenti mensili

## Descrizione delle tabelle

### Categorie

Contiene le 5 macro-categorie di agenti AI disponibili nel catalogo.

| Campo          | Tipo di Dato | Vincoli e Note              |
|----------------|--------------|-----------------------------|
| id_categoria   | INT          | PRIMARY KEY, AUTO_INCREMENT |
| nome_categoria | VARCHAR(100) | NOT NULL, UNIQUE            |

Il vincolo UNIQUE su nome\_categoria serve perchè senza di esso sarebbe possibile inserire due righe 'Agenti di Marketing', corrompendo la navigazione del catalogo.

### Utenti

Memorizza tutti gli account del sistema di clienti e amministratori. Il campo ruolo determina i permessi di accesso all'intera applicazione.

| Campo              | Tipo di Dato           | Vincoli e Note                               |
|--------------------|------------------------|----------------------------------------------|
| id_utente          | INT                    | PRIMARY KEY, AUTO_INCREMENT                  |
| nome               | VARCHAR(100)           | /                                            |
| cognome            | VARCHAR(100)           | /                                            |
| email              | VARCHAR(150)           | UNIQUE                                       |
| password           | VARCHAR(255)           | Hash bcrypt generato da password_hash()      |
| ruolo              | ENUM('utente','admin') | NOT NULL, DEFAULT 'utente'                   |
| data_registrazione | DATE                   | valorizzata con NOW() al momento dell'INSERT |

Il campo password è dichiarato VARCHAR(255) perché l'hash bcrypt generato da PHP occupa tra 60 e 72 caratteri e il margine garantisce compatibilità con eventuali futuri algoritmi più lunghi.

## Prodotti

Contiene i 15 piani di abbonamento: 3 per ognuna delle 5 categorie, uno per ciascuna fascia (base, pro, enterprise).

| Campo              | Tipo di Dato                    | Vincoli e Note                                                    |
|--------------------|---------------------------------|-------------------------------------------------------------------|
| id_prodotto        | INT                             | PRIMARY KEY, AUTO_INCREMENT                                       |
| nome_prodotto      | VARCHAR(150)                    | NOT NULL, UNIQUE                                                  |
| descrizione        | VARCHAR(500)                    | Testo descrittivo del piano, visualizzato nelle card del catalogo |
| prezzo             | DECIMAL(10,2)                   | NOT NULL                                                          |
| fascia_abbonamento | ENUM('base','pro','enterprise') | NOT NULL                                                          |
| id_categoria       | INT                             | FOREIGN KEY → categorie(id_categoria) con integrità referenziale  |

DECIMAL(10,2) è preferito a FLOAT per i valori monetari perché FLOAT può introdurre imprecisioni binarie. La fascia\_abbonamento come ENUM impedisce l'inserimento di valori non previsti.

## Carte\_salvate

Gestisce i metodi di pagamento memorizzati dagli utenti per semplificare gli acquisti successivi.

| Campo        | Tipo di Dato                     | Vincoli e Note                                                |
|--------------|----------------------------------|---------------------------------------------------------------|
| id_carta     | INT                              | PRIMARY KEY, AUTO_INCREMENT                                   |
| id_utente    | INT                              | NOT NULL, FOREIGN KEY → utenti(id_utente)                     |
| intestatario | VARCHAR(100)                     | NOT NULL                                                      |
| numero_carta | VARCHAR(20)                      | NOT NULL                                                      |
| scadenza     | VARCHAR(5)                       | NOT NULL                                                      |
| circuito     | ENUM('visa','mastercard','amex') | NOT NULL                                                      |
| predefinita  | TINYINT(1)                       | NOT NULL, DEFAULT 0 → se 1: carta preselezionata al pagamento |

Il numero carta viene mascherato prima dell'INSERT. Tramite le funzioni preg\_replace() e substr() si elimina ogni spazio e si prende la sottostringa degli ultimi 4 caratteri e si sostituisce con \*\*\*\*.

Il campo predefinita di tipo TINYINT(1) funziona come booleano MySQL: 0=false, 1=true.

## Ordini

Registra ogni transazione di acquisto effettuata.

| Campo             | Tipo di Dato               | Vincoli e Note                                                               |
|-------------------|----------------------------|------------------------------------------------------------------------------|
| id_ordine         | INT                        | PRIMARY KEY, AUTO_INCREMENT                                                  |
| id_utente         | INT                        | FOREIGN KEY → utenti(id_utente)                                              |
| id_carta          | INT                        | FOREIGN KEY → carte_salvate(id_carta)                                        |
| data_acquisto     | DATE                       | NOT NULL                                                                     |
| data_rinnovo      | DATE                       | NOT NULL                                                                     |
| stato_abbonamento | ENUM('attivo','annullato') | NOT NULL, DEFAULT 'attivo', cambia in 'annullato' solo su richiesta utente   |
| totale_ordine     | DECIMAL(10,2)              | Somma dei prezzo_unitario dei prodotti nel carrello al momento dell'acquisto |

La logica di stato reale dell'abbonamento viene calcolata così: un ordine con stato='attivo' ma data\_rinnovo < CURDATE() viene mostrato come 'Scaduto' nell'interfaccia, ma non nel db. Questo è un possibile miglioramento futuro.

## Dettagliordini

Storicizza il prezzo al momento dell'acquisto, garantendo correttezza storica.

| Campo           | Tipo di Dato  | Vincoli e Note                      |
|-----------------|---------------|-------------------------------------|
| id_dettaglio    | INT           | PRIMARY KEY, AUTO_INCREMENT         |
| id_ordine       | INT           | FOREIGN KEY → ordini(id_ordine)     |
| id_prodotto     | INT           | FOREIGN KEY → prodotti(id_prodotto) |
| prezzo_unitario | DECIMAL(10,2) | NOT NULL                            |

Il campo prezzo\_unitario è fondamentale: se il prezzo di un piano AI dovesse cambiare nel futuro, tutti gli ordini passati mostrerebbero correttamente l'importo originariamente pagato, non il prezzo attuale. La quantità ordinata non è presente per scelta progettuale.

## Relazioni tra le Tabelle

Schema delle relazioni con cardinalità:

- utenti 1 a N ordini: un utente può effettuare più acquisti nel tempo, ciascuno registrato come ordine separato.
- utenti 1 a N carte\_salvate: un utente può memorizzare più carte di pagamento dove il campo predefinita indica quella principale.
- categorie 1 a N prodotti: ogni categoria contiene esattamente 3 prodotti (uno per fascia).

- ordini N a N prodotti creata tramite dettagliordini: un ordine può contenere prodotti di categorie diverse (es. AI Marketing Base + AI Analytics Pro) e un prodotto può comparire in ordini di utenti diversi.
- ordini N a 1 carte\_salvate: ogni ordine è associato a una carta specifica.

Tutte le FK sono definite con integrità referenziale: è impossibile inserire un ordine per un utente inesistente, o un dettaglio per un prodotto non nel catalogo. MySQL garantisce questa consistenza implicitamente.

## Popolamento del Database

Il file aiaiai\_store.sql contiene tutte le istruzioni CREATE TABLE e INSERT INTO per la creazione completa del database aiaiaistore. Il popolamento prevede:

| Tabella        | N. Righe | Note                                                                         |
|----------------|----------|------------------------------------------------------------------------------|
| categorie      | 5        | Agenti di Marketing, Analisi Dati, Automazione, Assistenza Clienti, Sviluppo |
| utenti         | 10       | 2 admin (Enrico Lanzarolo, Alberto Cappellari) + 8 utenti standard           |
| prodotti       | 15       | 3 per categoria (Base/Pro/Enterprise), prezzi da €9,99 a €399,99/mese        |
| carte_salvate  | 5        | Una per ciascun utente con ordini, numeri già mascherati                     |
| ordini         | 5        | Mix di stati: 3 attivi, 2 annullati, con date di rinnovo variate             |
| dettagliordini | 10       | Distribuiti tra i 5 ordini                                                   |

Le password nel file SQL sono già hashate. I valori in chiaro sono commentati nell'sql per il testing del sistema e sono i seguenti:

| Utente             | Email                             | Password (in chiaro) | Ruolo  |
|--------------------|-----------------------------------|----------------------|--------|
| Enrico Lanzarolo   | enrico.lanzarolo@aiaiaistore.it   | pass_enrico          | admin  |
| Alberto Cappellari | alberto.cappellari@aiaiaistore.it | pass_alberto         | admin  |
| Giovanni Centin    | giovanni.centin@gmail.com         | pass_giovanni        | utente |
| Sara Salenti       | sara.salenti@gmail.com            | pass_sara            | utente |
| Davide Bissoli     | davide.bissoli@gmail.com          | pass_davide          | utente |
| Giulia Mansardi    | giulia.mansardi@gmail.com         | pass_giulia          | utente |
| Andrea Giacomello  | andrea.giacomello@gmail.com       | pass_andrea          | utente |
| Chiara Schettino   | chiara.schettino@gmail.com        | pass_chiara          | utente |
| Matteo Damante     | matteo.damante@gmail.com          | pass_matteo          | utente |
| Elena Justremo     | elena.justremo@gmail.com          | pass_elena           | utente |

## Descrizione Script PHP

### • **connessione.php**

File di configurazione della connessione al database.

- Definisce 4 variabili di configurazione: `$host = 'localhost'`, `$username = 'root'`, `$password = ''`, `$dbase = 'aiaiaistore'`.
- La variabile `$conn` rimane disponibile nello scope globale di ogni script che include questo file.

### • **index.php**

Visualizza la home page e mostra il catalogo, gestisce anche l'interfaccia in base allo stato di autenticazione dell'utente.

- `session_start()`: avvia o riprende la sessione PHP corrente.
- `$loggato = isset($_SESSION['id_utente'])`: TRUE se l'utente ha una sessione attiva, FALSE altrimenti.
- Se `loggato`: personalizza il saluto nell'header, verifica conteggio del carrello, mostra pulsante 'i miei abbonamenti' e, se admin, mostra il bottone 'pannello admin'. Altrimenti mostra il bottone 'accedi'.
- Cinque card agenti: ogni card contiene un video MP4 in loop automatico, titolo categoria, sottotitolo tag, descrizione e pulsante 'Scopri di più'. Il link del pulsante è condizionale: se `loggato` porta a `php/catalogo.php?id_categoria=N` (dove N è 1-5), se non `loggato` porta a `php/login.php?cat=N`. Il parametro `?cat=N` permette di ricordare la destinazione desiderata anche dopo il login.

### • **login.php**

Gestisce l'autenticazione degli utenti registrati. Gestisce anche il passaggio del parametro categoria `?cat=N` per il redirect post-login alla destinazione scelta inizialmente dall'utente.

- `session_start()` + include `connessione.php`: avvio sessione e connessione database.
- Lettura parametro per redirect post-login `?cat: isset($_GET['cat']) ? (int)$_GET['cat'] : 0`. Il Get viene usato perché impedisce injection.
- Redirect se già `loggato`: se `$_SESSION['id_utente']` esiste, l'utente viene reindirizzato immediatamente.
- Verifica del metodo: accetta solo `request_method == POST`
- Sicurezza dei campi: tutti i campi vengono trattati con comandi specifici per prevenire SQL injection.
- La password NON viene trattata perché non viene mai inserita in SQL: viene usata solo con `password_verify()`.
- Query di ricerca utente per email per evitare doppioni: se `mysqli_num_rows() > 0` significa che esiste un account con quell'email.
- Creazione sessione: `$_SESSION['id_utente']` e `$_SESSION['ruolo']` vengono valorizzati con i dati del DB. Da questo momento l'utente è autenticato per tutte le pagine protette.

- Gestione errore: se l'email non viene trovata o la password è errata, viene valorizzata \$errore con messaggio generico.

## • registrazione.php

Crea un nuovo account utente. Convalida gli input tramite HTML, verifica l'unicità dell'email, esegue l'INSERT e autentica automaticamente l'utente senza richiedere un secondo login.

- Redirect se già loggato: se la sessione esiste, redirect immediato alla home
- Hashing password: password\_hash(\$\_POST['password'], PASSWORD\_DEFAULT) Controllo email duplicata
- INSERT utente
- Crea la sessione come fatto precedentemente in login.php
- Redirect post-registrazione: stessa logica del login

## • logout.php

Termina la sessione dell'utente e reindirizza alla home.

- session\_start(): necessario per accedere alla sessione esistente prima di distruggerla.
- session\_destroy(): elimina tutti i dati della sessione corrente
- header('Location: ../index.php'): redirect immediato alla home.

## • catalogo.php

Visualizza i tre piani di abbonamento di una categoria specifica. Mostra la disponibilità di ogni piano rispetto agli abbonamenti dell'utente. Questo significa che se l'utente ha già la versione Pro dell'agente non potrà comprare la versione base ma solo migliorare alla versione Enterprise.

- Protezione autenticazione: if (!isset(\$\_SESSION['id\_utente'])) → redirect a login.php. Serve per rendere questa pagina non accessibile senza sessione attiva.
- Query ordini attivi: recupera tutti gli id\_ordine dell'utente con stato\_abbonamento='attivo' AND data\_rinnovo >= CURDATE(). La condizione CURDATE() filtra gli ordini tecnicamente attivi ma temporalmente scaduti.
- Array lista\_ordini: se ci sono ordini, gli id vengono accumulati in un array PHP con while + mysqli\_fetch\_assoc().
- implode(',', \$lista\_ordini): converte l'array in stringa CSV per usarla nella query successiva
- Query dettagliordini: recupera gli id\_prodotto associati agli ordini attivi trovati.

## • prodotto.php

Pagina di dettaglio del singolo prodotto. Mostra tutte le caratteristiche del piano selezionato, l'elenco delle feature incluse e la box acquisto con stato

- Protezione autenticazione: redirect a login.php se sessione assente.
- Query prodotto con JOIN. recupera in una sola query tutti i dati del prodotto più il nome e l'id della sua categoria, evitando una query aggiuntiva.

- Doppia verifica risultato: si controlla sia il ritorno booleano di `mysqli_query()` che il numero di righe restituite. Se il prodotto non esiste, redirect alla home.

## • carrello.php

Gestisce tutte le operazioni sul carrello: aggiunta prodotti, rimozione, visualizzazione contenuto, calcolo totale e raccolta dati di pagamento prima dell'invio a `conferma_ordine.php`.

- Il carrello è implementato come array PHP in sessione: `$_SESSION['carrello']` è un array di interi (ID prodotto).
- Inizializzazione sicura: `if (!isset($_SESSION['carrello']) || !is_array($_SESSION['carrello'])) { $_SESSION['carrello'] = []; }`. Garantisce che il carrello sia sempre un array valido prima di qualsiasi operazione.
- `in_array($id_da_aggiungere, $_SESSION['carrello'])`: verifica che il prodotto non sia già presente. Impedisce duplicati nel carrello (non ha senso avere lo stesso piano AI due volte).
- `$_SESSION['carrello'][] = $id_da_aggiungere`: aggiunta in append all'array.
- `array_search($id_da_rimuovere, $_SESSION['carrello'])`: cerca il valore nell'array e restituisce la sua chiave numerica (posizione). Restituisce FALSE se non trovato.
- `unset($_SESSION['carrello'][$chiave])`: rimuove l'elemento alla posizione trovata.
- Redirect a `carrello.php`: ricarica la pagina pulita dopo la rimozione.
- Calcolo totale: `$totale += $prod['prezzo']` accumulato ad ogni iterazione.
- Query carte salvate: `SELECT * FROM carte_salvate WHERE id_utente = '$id_utente'`. Recupera tutte le carte dell'utente per mostrarle come opzioni radio nel form di pagamento.

## Validazione JavaScript

- Ascolto evento submit: `document.getElementById('formPagamento').addEventListener('submit', function(e) {...})`.
- Se il metodo selezionato è 'carta\_salvata': nessuna validazione aggiuntiva, i dati sono già nel DB.
- Se il metodo è 'nuova\_carta': 4 validazioni in cascata con `e.preventDefault()` che blocca il submit in caso di errore.

## • conferma\_ordine.php

- Implementa un flusso di conferma in due fasi distinte per prevenire acquisti accidentali. Gestisce anche l'inserimento della nuova carta, l'ordine e tutti i dettagli nel database.
- Protezione autenticazione: redirect se sessione assente.
- Verifica metodo HTTP
- Verifica carrello non vuoto
- `$metodo = $_POST['metodo_pagamento']`: può essere 'carta\_salvata' o 'nuova\_carta'.
- Se `$errore` valorizzato: redirect a `carrello.php?errore=urlencoded_message` e exit.
- `SELECT id_prodotto, prezzo, nome_prodotto FROM prodotti WHERE id_prodotto IN ($ids)`: recupera i dati di tutti i prodotti nel carrello in una sola query. Il totale viene accumulato sommando i prezzi nel loop while.

## Fase 1

- Viene renderizzata una pagina HTML con il riepilogo dell'ordine: lista prodotti con prezzi, totale, metodo di pagamento e numero carta.
- Il link 'Rifiuta e torna al carrello' permette di annullare senza conseguenze.
- exit dopo la chiusura di mysqli: il file PHP termina qui per il primo POST. Il secondo POST (con conferma\_definitiva) riprende dall'inizio dello script e supera il blocco isset.

## Fase 2

- Se metodo è 'nuova\_carta': vengono controllati tutti i parametri
- INSERT in ordini
- mysqli\_insert\_id(\$conn) dopo l'INSERT ordine: recupera l'id\_ordine per i dettagli.
- Loop INSERT in dettagliordini: per ogni prodotto in \$prodotti\_mostra esegue un INSERT separato con id\_ordine, id\_prodotto e prezzo\_unitario.
- Svuotamento carrello: unset(\$\_SESSION['carrello']) + \$\_SESSION['carrello'] = []. Il doppio passaggio garantisce che l'array esista ma sia vuoto.
- SELECT carta per la ricevuta: recupera i dati della carta usata per mostrarli nella pagina di conferma finale.

### • storico\_ordini.php

Visualizza lo storico completo degli abbonamenti dell'utente con stato aggiornato in tempo reale e offre la possibilità di annullare gli abbonamenti attivi.

- Query ordini: `SELECT * FROM ordini WHERE id_utente = '$id_utente' ORDER BY data_acquisto DESC`. Recupera tutti gli ordini dell'utente ordinati dal più recente al più vecchio.
- if (isset(\$\_GET['annullato'])): se annulla\_abbonamento.php ha reindirizzato con ?annullato=1, viene mostrato il div .msg-ok con il messaggio di conferma. Il messaggio scompare al prossimo accesso senza il parametro.
- Colonne: N° ordine, Agente/i, Data acquisto, Rinnovo/Scadenza, Carta, Totale/mese, Stato, Azione.
- Per ogni riga: query separata per i dati della carta (`SELECT numero_carta, circuito FROM carte_salvate WHERE id_carta = '$id_carta_corrente'`) — commentato come 'senza JOIN' per coerenza procedurale.
- Query prodotti dell'ordine: `SELECT p.nome_prodotto FROM dettagliordini d JOIN prodotti p ON d.id_prodotto = p.id_prodotto WHERE d.id_ordine = '...'` — restituisce i nomi di tutti i prodotti nell'ordine, uniti con implode(', ', ...) per la visualizzazione.

### • annulla\_abbonamento.php

Riceve via POST la richiesta di annullamento di un abbonamento, esegue tutti i controlli di sicurezza necessari e aggiorna lo stato nel database.

- Lettura id\_ordine: `isset($_POST['id_ordine']) ? (int)$_POST['id_ordine'] : 0`.
- SELECT di verifica tripla: `WHERE id_ordine = '$id_ordine' AND id_utente = '$id_utente' AND stato_abbonamento = 'attivo' AND data_rinnovo >= CURDATE()`. Questo singolo WHERE

garantisce simultaneamente: a) l'ordine esiste, b) appartiene all'utente corrente (ownership), c) è nello stato 'attivo', d) non è già scaduto per data. Se anche una sola condizione fallisce, `mysqli_num_rows() == 0` e lo script termina senza aggiornare nulla.

- UPDATE: SET stato\_abbonamento = 'annullato' WHERE id\_ordine = '\$id\_ordine' AND id\_utente = '\$id\_utente'. Il secondo vincolo sull'utente nella WHERE è una ridondanza di sicurezza: anche se la SELECT precedente è già un controllo sufficiente, l'UPDATE include lo stesso filtro.
- Redirect: header('Location: storico\_ordini.php?annullato=1'). Il parametro ?annullato=1 attiva il flash message di conferma nella pagina storico.

## • index\_admin.php

Pannello di controllo riservato agli amministratori. Mostra KPI business aggregati, distribuzione degli abbonamenti per fascia e gli ultimi utenti registrati.

- Tot. utenti: SELECT COUNT(\*) AS n FROM utenti WHERE ruolo = 'utente'. Conta solo gli utenti standard, non gli admin.
- Tot. ordini: SELECT COUNT(\*) AS n FROM ordini. Conta tutti gli ordini senza filtri di stato.
- Abbonamenti attivi: SELECT COUNT(\*) AS n FROM ordini WHERE stato\_abbonamento = 'attivo' AND data\_rinnovo >= CURDATE(). Usa CURDATE() per filtrare anche quelli scaduti per data.
- Fatturato mensile: SELECT SUM(totale\_ordine) AS tot FROM ordini WHERE stato\_abbonamento = 'attivo' AND data\_rinnovo >= CURDATE(). Somma i totali degli abbonamenti effettivamente attivi
- Volume storico: SELECT SUM(totale\_ordine) AS tot FROM ordini. Somma di tutti gli ordini, inclusi annullati e scaduti
- SELECT nome, cognome, email, data\_registrazione FROM utenti WHERE ruolo = 'utente' ORDER BY data\_registrazione DESC LIMIT 6. Gli ultimi 6 utenti standard registrati.
- Rendering con while (mysqli\_fetch\_assoc): lista con nome completo, email e data formattata con date('d/m/Y', strtotime(...)).

## Sicurezza: Misure Implementate

Di seguito il riepilogo di tutte le misure di sicurezza adottate nel progetto:

| Misura        | Tecnica Usata                                                                                              | Dove è stata applicata                                                 |
|---------------|------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|
| SQL Injection | <code>mysqli_real_escape_string()</code> su stringhe, casting (int) su ID numerici                         | login, registrazione, conferma_ordine, tutte le query con input utente |
| XSS Output    | <code>htmlspecialchars()</code> su tutti i valori DB prima dell'output HTML                                | header, catalogo, prodotto, storico, conferma                          |
| Password      | <code>password_hash()</code> per salvataggio, <code>password_verify()</code> per confronto e mai in chiaro | registrazione, login                                                   |

| Misura                  | Tecnica Usata                                                               | Dove è stata applicata                       |
|-------------------------|-----------------------------------------------------------------------------|----------------------------------------------|
| Numero carta            | Mascheratura ****XXXX server-side con preg_replace+substr prima dell'INSERT | conferma_ordine, lato server                 |
| Ownership ordini        | WHERE id_utente = '\$id_utente' in ogni query che agisce su dati utente     | annulla_abbonamento, conferma_ordine carta   |
| Controllo metodo HTTP   | \$_SERVER['REQUEST_METHOD'] != 'POST' → redirect                            | conferma_ordine, annulla_abbonamento         |
| Accesso pagine protette | Controllo sessione all'inizio di ogni script — redirect se assente          | catalogo, prodotto, carrello, storico, admin |
| Accesso admin           | Verifica ruolo='admin' in aggiunta al controllo sessione                    | index_admin.php                              |
| Validazione carte JS    | Regex + Date check client-side prima del submit del form                    | carrello.php, JavaScript                     |

## Considerazioni Finali

### Risultati Ottenuti

Il progetto AIAIAI STORE è stato completato con tutte le funzionalità base richieste dalla traccia, adattate coerentemente al contesto dei prodotti digitali ad abbonamento. Il sito è pienamente funzionante e copre l'intero flusso utente. Inoltre, il pannello admin offre una visione d'insieme business. Il design dark-mode è responsive per dispositivi mobile e desktop.

### Scelte Progettuali Motivate

- Carrello in sessione invece che in DB: per prodotti digitali con acquisto immediato, la sessione è sufficiente, più semplice e non richiede una tabella aggiuntiva.
- Mascheratura numero carta server-side: per conformità con le pratiche di sicurezza standard, il numero completo non viene mai inserito nel DB.
- Stato abbonamento calcolato a runtime in PHP: evita la necessità di un cron job per aggiornare i record scaduti
- ORDER BY FIELD() per priorità fascia: permette di recuperare con una sola query il piano più alto posseduto in una categoria, senza logica PHP post-query.
- CSS sviluppato con supporto prevalente dell'AI: HTML e PHP sono stati progettati autonomamente con aiuti nelle sezioni più complicate; l'AI è stata usata per ottimizzare efficienza, indentazione e per la generazione degli stili CSS avanzati.

## Difficoltà Incontrate

- Hashing bcrypt: Il capire il funzionamento e la logica ha richiesto più tempo del previsto
- Gestione sessioni: le sessioni erano state viste in classe ma non in modo così approfondito e dunque ha richiesto sperimentazione diretta.
- Adattamento alla traccia: volendo cambiare contesto del negozio online per fare qualcosa più incentrato sugli abbonamenti invece che sul classico e-commerce pensare a come ristrutturare db e la traccia è stato dispendioso in termini di tempo.

## Possibili Miglioramenti Futuri

- Garantire una protezione SQL injection strutturalmente più solida.
- Rinnovo automatico degli abbonamenti via cron job: aggiornare data\_rinnovo e gestire pagamenti ricorrenti in modo automatizzato
- Gestione prodotti admin: form CRUD nel pannello admin per aggiungere, modificare ed eliminare prodotti e categorie senza accesso diretto al DB.
- Reset password via email: sistema di recupero password con token temporaneo inviato all'email registrata.
- Gateway di pagamento reale: integrazione con Stripe o PayPal per la gestione dei pagamenti effettivi.